

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab																		
Academic Year:2025-202																				
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week10 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number:19.3(Present assignment number)/24(Total number of assignments)																				
	Question		Expected Time to complete																	

		ete
1	Lab 19 – Code Translation: Converting Between Programming Languages Lab Objectives: • Understand how AI tools can assist in translating code between different programming languages. • Learn to verify correctness and functionality after translation. • Explore syntactic and semantic differences between languages (e.g., Python, Java, C++). • Practice debugging and optimizing AI-translated code.	Week 10 - Wednesday
	Task 1 – Python to C++ Conversion Task: Translate a simple Python class into C++ using AI assistance. Instructions: • Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code • Ensure proper handling of: ◦ Constructors ◦ Data types ◦ Access specifiers • Compile and run both versions to verify output consistency Expected Output: An equivalent working C++ class translated from Python. Prompt Prompt Convert the following Python class into an equivalent C++ class. The class represents a Student with attributes name and age, and a method display(). Ensure proper use of constructors, data types, and access specifiers in C++. Also include a main() function to test the class.	

AI ASST 19.3_Vjpyrb

File Edit View Insert Runtime Tools Help

commands + Code + Text Run all

Start coding or generate with AI.

C++ Equivalent of the Student Class

This C++ code defines a 'Student' class with 'name' (string) and 'age' (integer) as private member variables. It includes:

```
- **Constructor:** Initializes the 'name' and 'age' of a 'Student' object.
- **display() method:** Prints the student's name and age.
- **main() function:** Demonstrates how to create 'Student' objects and call their 'display()' method.
```

To compile and run this code, you would typically save it as a '.cpp' file (e.g., 'student.cpp') and use a C++ compiler like g++:

```
bash
g++ student.cpp -o student
./student
```

Python

Start file student.cpp

#include <iostream>

#include <string>

class Student {

private:

std::string name;

int age;

public:

Prompt Convert the following Python class into an equivalent C++ class. The class represents a Student with attributes name and age, and a method display(). Ensure proper use of constructors, data types, and access specifiers in C++. Also include a main() function to test the class.

Here's the C++ equivalent of your Python 'Student' class, including a

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

AI ASST 19.3_Vjpyrb

File Edit View Insert Runtime Tools Help

commands + Code + Text Run all

Writing student.cpp

Compile

Compile the C++ code

g++ student.cpp -o student

Run the compiled executable

./student

Displaying Student 1 Info:

Student Name: Alice

Student Age: 20

Displaying Student 2 Info:

Student Name: Bob

Student Age: 22

Here's the C++ equivalent of your Python 'Student' class, including a main() function to test it. I've ensured proper use of constructors, data types, and access specifiers.

It looks like there was a SyntaxError in the cells I just created. This happened because the markdown code fences (like ```python) were accidentally included in the executable code cells. I will fix this by removing those fences so the %writefile command and

What can I help you build?

Gemini 2.5 Flash

Explanation / Justification

- Python is **dynamic**, C++ is **static typed**
- Constructor:
 - Python → `__init__`
 - C++ → class constructor
- Access Specifiers:
 - Python → public by default
 - C++ → explicitly private / public
- Output:
 - Python → `print()`
 - C++ → `cout`

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java `isPrime()` method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

Prompt

Convert the following Java method `isPrime()` into Python.

Ensure Pythonic syntax and test with multiple inputs.

AI ASST 19.3_ViPyNb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
### Python Equivalent of 'isPrime()' Method

This Python function 'is_prime(n)' checks if a given integer 'n' is a prime number. It follows the same logic as the provided Java.

- Numbers less than or equal to 1 are not prime.
- It iterates from 2 up to the square root of 'n'. If any number in this range divides 'n' evenly, 'n' is not prime.
- If no such divisor is found, 'n' is prime.

'''python
def is_prime(n):
    if n <= 1:
        return False
    # Loop from 2 up to the square root of n
    # We only need to check divisibility up to sqrt(n)
    # because if n has a divisor greater than its square root,
    # it must also have a divisor smaller than its square root.
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
'''
```

Test Cases

The following code block tests the `is_prime()` function with vari

Provide the Java `isPrime()` method code

Here's the Python equivalent of your Java `isPrime()` method. I've translated the logic into a Pythonic function and included several test cases to demonstrate its usage.

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

AI ASST 19.3_ViPyNb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[5] print(f"is_prime(101): {is_prime(101)}") # Expected: True
✓ print(f"is_prime(97): {is_prime(97)}") # Expected: True

is_prime(0): False
is_prime(1): False
is_prime(2): True
is_prime(3): True
is_prime(4): False
is_prime(7): True
is_prime(9): False
is_prime(13): True
is_prime(25): False
is_prime(101): True
is_prime(97): True
```

Provide the Java `isPrime()` method code

Here's the Python equivalent of your Java `isPrime()` method. I've translated the logic into a Pythonic function and included several test cases to demonstrate its usage.

Working...

Explanation

- Java uses `boolean`, Python uses `True/False`

- Loop conversion:
 - Java → for(int i=...)
 - Python → range()
- Cleaner syntax in Python (no semicolons, no type declarations)

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

Task 4 – SQL to Pandas Que

Prompt

Convert the following bubble sort pseudocode into Python.

Ensure the code is executable and test with a sample list.

Pseudocode:

```
FOR i from 0 to n-1
  FOR j from 0 to n-i-1
    IF arr[j] > arr[j+1]
      SWAP arr[j], arr[j+1]
```

AI ASST 19.3_Vjpynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
## Python Implementation of Bubble Sort

This Python function 'bubble_sort(arr)' sorts a list of numbers in ascending order using the bubble sort algorithm. The logic dir

- **Outer loop ('i')**: Controls the number of passes through the list. With each pass, the largest unsorted element "bubbles" to the end of the list.
- **Inner loop ('j')**: Compares adjacent elements and swaps them if they are in the wrong order. The range of this loop decreases by one after each pass.

'''python
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n-i-1):
            # Swap if the element found is greater
            # than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Test Case

The following code block defines a sample list and then applies the bubble sort function to it.

Prompt: Convert the following bubble sort pseudocode into Python. Ensure the code is executable and test with a sample list. Pseudocode: FOR i from 0 to n-1 FOR j from 0 to n-i-1 IF arr[j] > arr[j+1] SWAP arr[j], arr[j+1]

Certainly! Here's the Python implementation of the bubble sort algorithm based on your pseudocode. I've also included a simple test case to demonstrate how it works.

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

AI ASST 19.3_Vjpynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
(4)
def bubble_sort(arr):
    sorted_list_2 = bubble_sort(sample_list_2)
    print(f"Sorted list 2: {sorted_list_2}")

# Edge case: already sorted
sample_list_3 = [1, 2, 3, 4, 5]
print(f"Original list 3 (already sorted): {sample_list_3}")
sorted_list_3 = bubble_sort(sample_list_3)
print(f"Sorted list 3: {sorted_list_3}")

# Edge case: reverse sorted
sample_list_4 = [5, 4, 3, 2, 1]
print(f"Original list 4 (reverse sorted): {sample_list_4}")
sorted_list_4 = bubble_sort(sample_list_4)
print(f"Sorted list 4: {sorted_list_4}")
```

Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
Original list 2: [5, 1, 4, 2, 8]
Sorted list 2: [1, 2, 4, 5, 8]
Original list 3 (already sorted): [1, 2, 3, 4, 5]
Sorted list 3: [1, 2, 3, 4, 5]
Original list 4 (reverse sorted): [5, 4, 3, 2, 1]
Sorted list 4: [1, 2, 3, 4, 5]

Certainly! Here's the Python implementation of the bubble sort algorithm based on your pseudocode. I've also included a simple test case to demonstrate how it works.

The Python 'bubble_sort()' function has been successfully executed with the provided test cases. Here are the results:

Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]

What can I help you build?

Gemini 2.5 Flash

Explanation

- Nested loops directly map from pseudocode
- Python allows **tuple swapping**:

`a, b = b, a`

- Time Complexity: **$O(n^2)$**

ry

Task:

Translate a **SQL query** into a **Pandas DataFrame operation** in Python.

Instructions:

- Provide AI with a SQL query (e.g., `SELECT name, salary FROM employees WHERE salary > 50000`).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Prompt

Convert the following SQL query into a Pandas DataFrame operation:

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

Also create a sample DataFrame and test it.

AI ASST 19.3_Viptynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
## SQL to Pandas Conversion: Filtering and Selecting

This section demonstrates how to convert the SQL query 'SELECT name, salary FROM employees WHERE salary > 50000;' into a Pandas DataFrame operation.

First, a sample DataFrame named 'employees_df' is created to simulate an 'employees' table with 'name', 'age', and 'salary' columns.

Then, the Pandas operation filters the 'employees_df' to include only employees with a 'salary' greater than 50000 and selects the
```

Import pandas as pd

```
# Create a sample DataFrame resembling an 'employees' table
employees_df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'age': [30, 24, 35, 40, 28],
    'salary': [60000, 45000, 75000, 50000, 62000]
})

print("Original DataFrame:")
display(employees_df)

# Convert the SQL query 'SELECT name, salary FROM employees WHERE salary > 50000;' to Pandas
# 1. Filter rows where 'salary' > 50000
# 2. Select 'name' and 'salary' columns
result_df = employees_df[employees_df['salary'] > 50000][['name', 'salary']]

print("\nResulting DataFrame (salary > 50000):")
display(result_df)
```

Prompt Convert the following SQL query into a Pandas DataFrame operation: SELECT name, salary FROM employees WHERE salary > 50000; Also create a sample DataFrame and test it.

Certainly! Here's how you can perform the SQL query 'SELECT name, salary FROM employees WHERE salary > 50000;' using Pandas DataFrame operations, along with a sample DataFrame for testing.

Accept & Run Accept X Cancel

What can I help you build?

Gemini 2.5 Flash

AI ASST 19.3_Viptynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
# Convert the SQL query 'SELECT name, salary FROM employees WHERE salary > 50000;' to Pandas
# 1. Filter rows where 'salary' > 50000
# 2. Select 'name' and 'salary' columns
result_df = employees_df[employees_df['salary'] > 50000][['name', 'salary']]

print("\nResulting DataFrame (salary > 50000):")
display(result_df)
```

Original DataFrame:

	name	age	salary
0	Alice	30	60000
1	Bob	24	45000
2	Charlie	35	75000
3	David	40	50000
4	Eve	28	62000

Resulting DataFrame (salary > 50000):

	name	salary
0	Alice	60000
2	Charlie	75000
4	Eve	62000

Next steps: Generate code with employees_df New interactive shell

Prompt Convert the following SQL query into a Pandas DataFrame operation: SELECT name, salary FROM employees WHERE salary > 50000; Also create a sample DataFrame and test it.

Certainly! Here's how you can perform the SQL query 'SELECT name, salary FROM employees WHERE salary > 50000;' using Pandas DataFrame operations, along with a sample DataFrame for testing.

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Explanation

- SQL → SELECT columns → [['col1', 'col2']]
- SQL → WHERE condition → boolean filtering
- Pandas uses **DataFrame slicing**

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
 - Syntax differences
 - Library support
 - Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

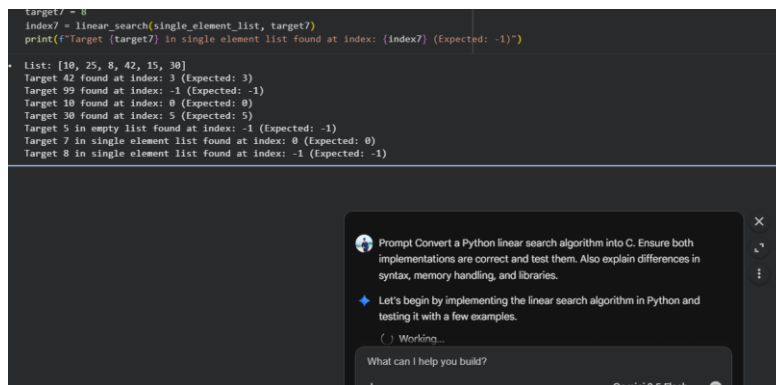
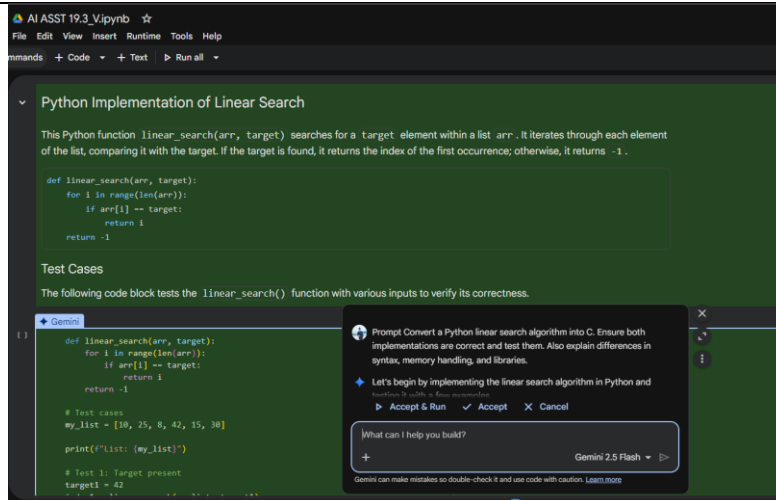
Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

Prompt

Convert a Python linear search algorithm into C.

Ensure both implementations are correct and test them.

Also explain differences in syntax, memory handling, and libraries.



Explanation / Challenges

◇ Syntax Differences

- Python → simple, readable
- C → strict syntax, semicolons

◇ Memory Management

- Python → automatic
- C → manual (arrays, pointers)

◇ Libraries

- Python → built-in functions
- C → requires headers (`stdio.h`)